

PINAKA Application - Interview Quick Revision

One-page style revision sheet

30-Second Explanation

In PINAKA, **pke::Application** is an abstract-style reusable engine base class that owns the application lifecycle and main loop. Pinaka Studio defines **StudioApplication**, which derives from it and overrides a virtual `update()` hook. The base class keeps the running state private and exposes `requestShutdown()` so derived applications can request termination without directly modifying internal state.

Architecture

```
PinakaStudio.exe main() | v StudioApplication | | public inheritance v pke::Application
[PinakaEngine.dll] | v run() -> init -> loop(update) -> shutdown
```

Key API Decisions

Member	Why
<code>run()</code> - public	External code starts the application lifecycle.
<code>update()</code> - protected virtual	Derived applications customize per-frame behavior; base owns the loop.
<code>requestShutdown()</code> - protected	Derived application can request termination without accessing private state.
<code>initEngine()/shutdown()</code> - private	Internal lifecycle implementation should not be arbitrarily called by Studio.
<code>m_Running</code> - private	Application owns its lifecycle state; demonstrates encapsulation.
virtual destructor	Safe destruction when a derived object is deleted through an Application pointer.
PK_API on Application	Application is part of the engine DLL API; StudioApplication is not.

Likely Interview Questions

Why not put the loop in Studio?

Because the engine should provide and control the common application lifecycle. Client applications customize hooks instead of reimplementing engine infrastructure.

Why is `update()` virtual?

`Application::run()` can invoke derived application behavior through runtime polymorphism.

Why protected instead of public?

`update()` and `requestShutdown()` are intended as derived-class facilities, not general operations for arbitrary external callers.

Why keep `m_Running` private?

To preserve encapsulation and lifecycle invariants. Derived code requests a state transition rather than modifying engine internals directly.

Why a virtual destructor?

If `StudioApplication` is destroyed through a `pke::Application` pointer, the derived destructor must run before the base destructor.

What design pattern is this similar to?

Template Method: the base class defines the lifecycle algorithm while derived classes override selected

steps/hooks.

Application vs Engine vs Studio?

Studio is the executable/product. Engine is reusable technology. Application is the engine-provided lifecycle abstraction used by a concrete client such as Studio.

Remember This Flow

```
main() -> StudioApplication -> Application::run() -> initEngine() -> while (m_Running) -> virtual  
update() -> requestShutdown() makes m_Running false -> shutdown() -> return to main()
```

Current Limitation / Next Step

The current Studio update calls requestShutdown() only as a test. In the real architecture, a **Window Close Event** will cause the shutdown request. The next subsystem is therefore the Window abstraction, followed naturally by Events.

Interview One-Liner

"I designed the engine Application base class to own the lifecycle and main loop, while derived clients such as the editor customize per-frame behavior through virtual hooks; internal running state stays encapsulated and shutdown is requested through a controlled API."